The Journal of Open Source Software

Jangular: A Full-Stack Framework for Java and Angular Integration

Nathan Aldyth Prananta Ginting [1]

1 School of Engineering and Technology, Sunway University, Malaysia

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 03 May 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Modern enterprise applications increasingly demand seamless integration between robust backend technologies like Java (Spring Boot) and dynamic frontend frameworks like Angular. However, this combination often results in fragmented workflows, steep learning curves, and increased development overhead—particularly for indie or solo developers aiming to build scalable, enterprise-grade solutions. This paper introduces Jangular, a conceptual framework designed to unify Java and Angular development through standardized architecture, developer-friendly tooling, and streamlined project scaffolding. Inspired by the cohesive philosophies of frameworks like Laravel (Laravel Team, 2025a, 2025b; Thompson, 2024) and modern monorepo practices (AngularMinds Team, 2023; Hauck, 2025), Jangular aims to improve productivity, maintainability, and developer experience. The framework proposes integrated support for Dockerized environments (Kodama, 2024; Squillace, 2018), authentication, CLI scaffolding, and modular architecture, making it easier for developers to build full-stack applications that are production-ready from day one.

Statement of Need

Enterprise web application development often involves using Java-based frameworks (particularly Spring Boot) for backend services and Angular for frontend interfaces. While both technologies are robust and feature-rich individually, their integration presents numerous challenges that impact developer productivity and application maintainability (JavaNexus, 2025).

Current approaches to integrating Java and Angular typically require developers to:

- Manually configure separate build processes and deployment pipelines
- Resolve cross-origin resource sharing (CORS) issues between the backend and frontend
- Navigate disconnected documentation and tooling ecosystems
- Manage complex project structures and versioning strategies across different runtimes
- Overcome steep learning curves in mastering both ecosystems simultaneously
- Work with redundant or misaligned authentication/authorization flows
- Duplicate environment configurations and environment variable management

This fragmentation not only affects development speed but also introduces greater risk of misconfiguration, security vulnerabilities, and inefficiencies in collaboration among teams. Without a unified framework or toolset, development teams must build and maintain their own glue code and workflows, diverting focus from delivering business value.

Jangular addresses these challenges by providing a cohesive framework that simplifies the integration of Java and Angular, making it particularly valuable for:

1. **Indie developers** who need to rapidly prototype and develop enterprise-grade applications

2. **Educational environments** where students need to focus on learning core concepts rather than configuration details
3. **Enterprise teams** seeking to standardize development practices and reduce onboarding time (Atlassian Team, 2024; GitHub Team, 2025)

Key Features

Jangular's design focuses on several key features that address the common pain points in Java and Angular integration:

Unified CLI Tool

Jangular provides a comprehensive command-line interface that combines the functionality of the Spring Boot CLI and Angular CLI, allowing developers to:

- Create new projects with standardized structure
- Generate compatible components, services, and models for both backend and frontend
- Run and test the application with a single command
- Deploy to various environments with built-in configurations

This approach draws inspiration from Laravel's Artisan console (Ayaz, 2024; Laravel Team, 2025a), which has demonstrated the productivity benefits of a powerful, integrated CLI tool for full-stack development.

Standardized Project Structure

The framework enforces a consistent monorepo architecture that organizes code in a logical and intuitive manner (AngularMinds Team, 2023; Hauck, 2025), promoting:

- Clear separation of concerns
- Easy navigation between related frontend and backend components
- Standardized naming conventions across the stack
- Simplified dependency management

Integrated Authentication

Jangular includes pre-configured authentication mechanisms that work seamlessly across the stack, supporting:

- JWT-based authentication
- OAuth 2.0 integration
- Role-based access control
- Secure session management

This approach is influenced by the success of authentication implementations in frameworks like RedwoodJS (RedwoodJS Team, 2025) and Blitz.js (Team, 2025).

Docker Integration

To simplify deployment and ensure consistency across environments, Jangular provides:

- Pre-configured Docker and Docker Compose files
- Environment-specific configurations
- Optimized container builds for both development and production
- Kubernetes deployment templates

These features align with modern container-native development practices (Kodama, 2024; Squillace, 2018).

DRAFT

Developer Experience Enhancements

Inspired by Laravel's focus on developer happiness (OfferZen Team, 2023), Jangular includes:

- Comprehensive documentation and interactive tutorials
- Intelligent error handling and debugging tools
- Hot-reloading capabilities for both Java and Angular
- Shared type definitions for seamless data transfer between layers

These enhancements directly address the increasing importance of developer experience in modern software development (Dallas, 2023; Gray, 2024; Pulumi Team, 2023).

Comparison with Existing Tools

While several tools address aspects of Java and Angular integration, Jangular differentiates itself through its focused approach and specific feature set:

Feature	Jangular	JHipster	Nx	NestJS + Angular
Backend	Java (Spring Boot)	Multiple options (JHipster team, 2025b)	Node.js (Nx Team, 2025c)	Node.js (Ducin, 2019; NestJS Team, 2025)
Frontend	Angular-focused	Multiple options (JHipster team, 2025a)	Multiple options (Nx Team, 2023, 2025b)	Angular (Omereshone, 2021)
Project Structure	Opinionated monorepo	Generated code (JHipster team, 2025a)	Flexible monorepo (Nx Team, 2025c)	Separate repositories
Learning Curve	Moderate	Steep	Moderate	Low (for Angular devs)
Docker Support	Built-in	Generator-based (JHipster team, 2025a)	Plugin-based	Manual config
Authentication	Integrated	Generated (JHipster team, 2025a)	Plugin-based (Nx Team, 2025a)	Manual integration
CLI Tools	Unified	Generator-based (JHipster team, 2025b)	Extensible (Nx Team, 2025b)	Separate CLIs

DRAFT

The Journal of Open Source Software

Architecture and Implementation

Jangular's architecture is designed to leverage the strengths of both Java and Angular while minimizing the friction points between them. The core components include:

1. **Jangular CLI**: A unified command-line tool that wraps the Spring Boot and Angular CLIs, providing seamless project creation, code generation, and development workflows.
2. **Project Template Engine**: A sophisticated templating system that generates standardized project structures with best practices baked in.
3. **Type Synchronization**: Automatic generation of TypeScript interfaces from Java models, ensuring type safety across the stack.
4. **Development Server**: An integrated development environment that runs both the Java backend and Angular frontend with hot reloading capabilities.
5. **Build Pipeline**: A unified build process that optimizes both the backend and frontend for production deployment, with built-in continuous integration support similar to JHipster's approach (JHipster team, 2025d).
6. **Documentation Generator**: Tools to automatically generate comprehensive API documentation from both Java and TypeScript code.
7. **Monitoring Tools**: Integrated monitoring capabilities for both backend and frontend components, inspired by JHipster's monitoring solutions (JHipster team, 2025c).

The initial implementation focuses on integrating Spring Boot and Angular, with plans to support additional Java frameworks like Micronaut in future releases.

Conclusion and Future Work

Jangular aims to bridge the gap between Java and Angular development by providing a cohesive, opinionated framework that addresses the common challenges of full-stack development. By prioritizing developer experience (Dallas, 2023; Humanitec Team, 2025; Pulumi Team, 2023) and standardized workflows, Jangular enables developers to focus on creating business value rather than managing technical integration details.

Future work includes:

- Developing a working prototype with core functionality
- Conducting user studies to validate the improvement in developer experience
- Exploring AI-assisted code generation for common patterns
- Extending support to additional Java frameworks
- Building a community-driven plugin ecosystem

Jangular represents a promising approach to addressing the challenges of full-stack development with Java and Angular. By prioritizing developer experience (Atlassian Team, 2024; Douglas, 2023) and embracing modern software development practices, this conceptual framework could significantly enhance productivity and code quality in enterprise environments.

References

AngularMinds Team. (2023). *A guide to running a full-stack angular application in a monorepo*. https://www.angularminds.com/blog/a-guide-to-running-a-full-stack-angular-application-in-a-monor

Atlassian Team. (2024). *New atlassian research on developer experience highlights a major disconnect between developers and leaders*. https://www.atlassian.com/blog/developer/developer-experience-report-2024

DRAFT

Ayaz, F. (2024). *How to create custom laravel commands with PHP artisan*. https://www.cloudways.com/blog/custom-artisan-commands-laravel/

Dallas, J. (2023). *What is developer experience & 17 ways to 10x your DX*. https://zeet.co/blog/developer-experience

Douglas, B. (2023). *How GitHub approaches internal developer experience*. https://devops.com/how-github-approaches-internal-developer-experience/

Ducin, T. (2019). *Nest.js, or angular and spring on the backend*. https://xtension.pl/en/nestjs-or-angular-and-spring-on-the-backend

GitHub Team. (2025). *Developer experience*. https://github.blog/tag/developer-experience/

Gray, L. (2024). *Developer experience (DevEx) trends shaping 2024*. https://www.gitkraken.com/blog/developer-experience-devex-trends-2024

Hauck, S. (2025). *Spring-angular-monorepo*. https://github.com/simonhauck/spring-angular-monorepo

Humanitec Team. (2025). *Internal developer portals: What you need to know*. https://humanitec.com/internal-developer-portal

JavaNexus. (2025). *Common spring boot & angular integration pitfalls*. https://javanexus.com/blog/spring-boot-angular-integration-pitfalls

JHipster team. (2025a). *Creating an application*. https://www.jhipster.tech/creating-an-app/

JHipster team. (2025b). *JHipster - full stack platform for the modern developer!* https://www.jhipster.tech/

JHipster team. (2025c). *Monitoring your JHipster applications*. https://www.jhipster.tech/monitoring/

JHipster team. (2025d). *Setting up continuous integration*. https://www.jhipster.tech/setting-up-ci/

Kodama, T. (2024). *What is "kubernetes native" CI/CD: History, key tools, pipeline and flow to the cloud native era*. https://www.nttdata.com/global/en/insights/focus/2024/what-is-kubernetes-native-cicd-history-key-tools-pipeline-and-flow-to-the-cloud-native-era

Laravel Team. (2025a). *Artisan console*. https://laravel.com/docs/12.x/artisan

Laravel Team. (2025b). *Blade templates*. https://laravel.com/docs/12.x/blade

NestJS Team. (2025). *NestJS - a progressive node.js framework*. https://nestjs.com/

Nx Team. (2023). *Scaffold angular applications with rspack*. https://nx.dev/blog/scaffold-angular-rspack-applications

Nx Team. (2025a). *Enterprise-grade security, built into the core*. https://nx.dev/enterprise/security

Nx Team. (2025b). *Nx and the angular CLI*. https://nx.dev/nx-api/angular/documents/nx-and-angular

Nx Team. (2025c). *Nx enterprise POV*. https://20.nx.dev/assets/enterprise/Nx-Enterprise-POV.pdf

OfferZen Team. (2023). *Taylor otwell on the importance of documentation and developer experience in laravel*. https://www.offerzen.com/blog/taylor-otwell-importance-of-documentation-and-deve

Omereshone, K. O. (2021). *An overview of typescript, angular and nest JS*. https://www.sololearn.com/blog/an-overview-of-typescript-angular-and-nest-js/

Pulumi Team. (2023). *Why developer experience (DevEx) is business critical*. https://www.

DRAFT

pulumi.com/blog/developer-experience-business-critical/

RedwoodJS Team. (2025). *Authentication*. https://docs.redwoodjs.com/docs/tutorial/chapter4/authentication/

Squillace, R. (2018). *5 reasons you should be doing container native development*. https://opensource.microsoft.com/blog/2018/04/23/5-reasons-you-should-be-doing-container-native-develop

Team, Blitz. js. (2025). *The fullstack react framework*. https://blitzjs.com/

Thompson, J. (2024). *Laravel commands: Top artisan commands to know and how to create them*. https://www.hostinger.com/uk/tutorials/laravel-commands

DRAFT